

rev-basic-0

- 문제 설명

이 문제는 사용자에게 문자열 입력을 받아 정해진 방법으로 입력값을 검증하여 correct 또는 wrong을 출력하는 프로그램이 주어집니다.

해당 바이너리를 분석하여 correct를 출력하는 입력값을 찾으세요!

획득한 입력값은 DH{} 포맷에 넣어서 인증해주세요.

- 풀이 과정

실제로 파일을 열어 실행한 결과 문제 설명과 맞게 잘못된 입력값을 넣은 결과 Wrong이라는 출력을 확인할 수 있다.

```
PS C:\Users\user> C:\Users\user\Downloads\2473fc0c-83fd-49a1-80e8-6f300497dfbf\chal0.exe
Input : aaaaaaa
Wrong
```

IDA에서 파일을 열어 문자열 검색으로 Correct를 찾은 뒤 상호 참조를 통해 main함수에서 이용되는 것을 알아낼 수 있었다.

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_140001190("Input : ", argv, envp);
7     sub_1400011F0("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

main함수를 보니 sub_14000100(v4)가 만족되면 Correct가 출력되는 것으로 추정된다.

sub_14000100(v4)함수 내부에서 Flag를 확인할 수 있다.

```
1 BOOL8 __fastcall sub_140001000(const char *a1)
2 {
3     return strcmp(a1, "Compar3_the_str1ng") == 0;
4 }
```

rev-basic-1

- 문제 설명

이 문제는 사용자에게 문자열 입력을 받아 정해진 방법으로 입력값을 검증하여 correct 또는 wrong을 출력하는 프로그램이 주어집니다.

해당 바이너리를 분석하여 correct를 출력하는 입력값을 알아내세요.

획득한 입력값은 DH{} 포맷에 넣어서 인증해주세요.

- 풀이 과정

rev-basic-0과 마찬가지로 아무 입력을 넣으면 Wrong이 출력된다.

```
PS C:\Users\user> C:\Users\user\Downloads\4482bbb1-7ec4-4a42-8325-d23253b84453\chal1.exe
Input : aaaa
Wrong
```

rev-basic-0과 같은 방법으로 IDA에서 Correct 문자열을 타고 main함수를 찾았다.

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_1400013E0("Input : ", argv, envp);
7     sub_140001440("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

main함수 확인 결과 sub_140001000(v4)를 만족하면 Correct인 것으로 보인다.

sub_140001000(v4) 내부로 들어가니 a1[0]부터 a1[20]까지 총 21개 문자를 검사하는 것을 확인할 수 있다. 이 문자를 이은 문자열이 Flag일 것으로 추측된다.

```
BOOL8 __fastcall sub_140001000(_BYTE *a1)
{
    if ( *a1 != 67 )
        return 0LL;
    if ( a1[1] != 111 )
        return 0LL;
    if ( a1[2] != 109 )
        return 0LL;
    if ( a1[3] != 112 )
        return 0LL;
    if ( a1[4] != 97 )
        return 0LL;
}
```

아래와 같은 C++ 코드를 만들고 아스키 값을 차례로 입력하니 Flag가 나왔다.

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int main() {
6      string answer;
7      char c;
8      int temp;
9      for(int i=0; i<21; i++) {
10         cin >> temp;
11         c = (char)temp;
12         answer += c;
13     }
14     cout << answer;
15     return 0;
16 }
17
```

rev-basic-2

- 문제 설명

이 문제는 사용자에게 문자열 입력을 받아 정해진 방법으로 입력값을 검증하여 correct 또는 wrong을 출력하는 프로그램이 주어집니다.

해당 바이너리를 분석하여 correct를 출력하는 입력값을 찾으세요!

획득한 입력값은 DH{} 포맷에 넣어서 인증해주세요.

- 풀이 과정

이전처럼 아무 입력이나 넣으면 Wrong이 출력된다.

```
PS C:\Users\user> C:\Users\user\Downloads\54f580d0-be5c-4f51-adf1-7684d6fc229f\chall2.exe
Input : eevee
Wrong
```

IDA에서 Correct 문자열을 따라 main함수에 도달했다. sub_140001000(v4)가 만족되면 Correct인 것으로 보인다.

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_1400011B0("Input : ", argv, envp);
7     sub_140001210("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

sub_140001000(v4)의 내부를 보니 18번에 걸쳐 aC[4*i]와 입력 문자열[i]를 비교해서 모두 같으면 1을 반환하는 것으로 보인다.

```
1 __int64 __fastcall sub_140001000(__int64 a1)
2 {
3     int i; // [rsp+0h] [rbp-18h]
4
5     for ( i = 0; (unsigned __int64)i < 0x12; ++i )
6     {
7         if ( *(_DWORD *)&aC[4 * i] != *(unsigned __int8 *)(a1 + i) )
8             return 0LL;
9     }
10    return 1LL;
11 }
```

해당 주소값으로 찾아가 Flag를 획득할 수 있었다.

```
C...o...m...p...
4...r...e..._...
t...h...e..._...
a...r...r...4...
y.....
```

rev-basic-3

- 문제 설명

생략

- 풀이 과정

아무 입력을 넣으면 당연히 Wrong이 출력된다.

```
PS C:\Users\user> C:\Users\user\Downloads\e20e0b3c-99e9-40ec-9ef8-4717939b883a\chall3.exe
Input : flareon
Wrong
```

이전과 마찬가지로 IDA에서 “Correct” 문자열을 타고 main함수에 도달했다. sub_140001000(v4)를 만족해야 Correct를 받을 수 있는 것으로 보인다.

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_1400011B0("Input : ", argv, envp);
7     sub_140001210("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

sub_140001000은 24번에 걸쳐 byte_140003000[i]와 입력값[i]+2*i를 i와 XOR한 값이 맞아야 1을 반환하는 것으로 분석했다.

```
1 __int64 __fastcall sub_140001000(__int64 a1)
2 {
3     int i; // [rsp+0h] [rbp-18h]
4
5     for ( i = 0; (unsigned __int64)i < 0x18; ++i )
6     {
7         if ( byte_140003000[i] != (i ^ *(unsigned __int8 *)(a1 + i)) + 2 * i )
8             return 0LL;
9     }
10    return 1LL;
11 }
```

XOR의 역연산도 XOR이다. 이를 이용해 주어진 식의 역연산을 문자열로 나타내는 C++코드를 아래와 같이 작성한 후 byte_140003000의 값을 차례로 입력해주니 Flag를 얻을 수 있었다.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 int main() {
5     string answer;
6     int num;
7     char temp;
8     for(int i=0; i<24; i++) {
9         cin >> hex >> num;
10        temp = (num - 2*i)^i;
11        answer += temp;
12    }
13    cout << answer;
14    return 0;
15 }
```


rev-basic-4

- 문제 설명
생략

- 풀이 과정
아무 입력을 넣으면 당연히 Wrong이 출력된다.

```
PS C:\Users\user> C:\Users\user\Downloads\eee2d203-e37c-492d-a0f3-7cb60fd1ba23\chall4.exe
Input : sylveon
Wrong
```

IDA에서 “Correct” 문자열을 타고 main함수에 도달하니 sub_140001000(v4)를 만족해야 하는 것으로 확인된다.

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_1400011C0("Input : ", argv, envp);
7     sub_140001220("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

sub_140001000내부를 확인하니 28번에 걸쳐 입력값과 i를 이용한 연산과 byte_140003000[i]이 일치해야 1을 반환하는 것으로 추정할 수 있다.

```
1 __int64 __fastcall sub_140001000(__int64 a1)
2 {
3     int i; // [rsp+0h] [rbp-18h]
4
5     for ( i = 0; (unsigned __int64)i < 0x1C; ++i )
6     {
7         if ( ((unsigned __int8)(16 * (_BYTE *) (a1 + i)) | ((int)*(unsigned __int8 *) (a1 + i) >> 4)) != byte_140003000[i] )
8             return 0LL;
9     }
10    return 1LL;
11 }
```

주어진 연산은 a1[i]에 16을 곱한 값, 즉 왼쪽으로 4비트 시프트한 값의 하위 8비트와 a1[i]를 오른쪽으로 4비트 시프트한 값의 or을 구하는 연산으로 결과적으로 상위 4비트와 하위 4비트가 서로 바뀌는 결과가 일어난다.
답을 구하기 위한 역연산은 같은 과정을 반복하면 다시 상위 4비트와 하위 4비트가 바뀌므로 원래대로 된다.

rev-basic-3의 코드를 재활용해서 아래와 같은 C++코드를 만들어 Flag를 찾을 수 있었다.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 int main() {
5     string answer;
6     int num;
7     char temp;
8     for(int i=0; i<28; i++) {
9         cin >> hex >> num;
10        temp = (num << 4) | (num >> 4);
11        answer += temp;
12    }
13    cout << answer;
14    return 0;
15 }
```

rev-basic-5

- 문제 설명
생략

- 풀이 과정
아무 입력을 넣으면 당연히 Wrong이 출력된다.

```
PS C:\Users\user> C:\Users\user\Downloads\b1146d44-99e0-45cb-ae74-25b3af81a949\chall5.exe
Input : eevee
Wrong
```

이전과 마찬가지로 IDA에서 main함수를 찾고 sub_140001000(v4)가 만족되어야 함을 파악했다.

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_1400011C0("Input : ", argv, envp);
7     sub_140001220("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

24번에 걸쳐 a1[i+1] + a1[i]가 byte_140003000[i]와 일치해야함을 알 수 있다.

```
1 __int64 __fastcall sub_140001000(__int64 a1)
2 {
3     int i; // [rsp+0h] [rbp-18h]
4
5     for ( i = 0; (unsigned __int64)i < 0x18; ++i )
6     {
7         if ( *(unsigned __int8 *)(a1 + i + 1) + *(unsigned __int8 *)(a1 + i) != byte_140003000[i] )
8             return 0LL;
9     }
10    return 1LL;
11 }
```

a1[24] + a1[23] == byte_140003000[23] == 0x00이고 unsigned니까 a1[24]과 a1[23]은 0일 수 밖에 없다.
a1[24], a1[23]이 모두 0인 것을 아니까 a1[i] = byte_140003000[i] - a1[i+1]으로 응용해 모든 a1[i]를 구할 수 있다.

아래와 같이 C++ 코드를 짰고 입력은 거꾸로 넣었더니 Flag를 획득할 수 있었다.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 int main() {
5     string answer;
6     int num;
7     char temp = 0;
8     answer += temp;
9     for (int i = 22; i >= 0; i--) {
10         cin >> hex >> num;
11         temp = (char)(num - answer[0]);
12         answer = temp + answer;
13     }
14     cout << answer;
15     return 0;
16 }
```

- 문제 설명
생략

아무 입력이 없으면 당연히 Wrong이 출력된다.

```
PS C:\Users\user> C:\Users\user\Downloads\a2aeb33c-d25e-40ac-8e8c-373426d40b32\chall6.exe
Input : eevee
Wrong
```

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_1400011B0("Input : ", argv, envp);
7     sub_140001210("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

```

1 __int64 __fastcall sub_140001000(__int64 a1)
2 {
3     int i; // [rsp+0h] [rbp-18h]
4
5     for ( i = 0; (unsigned __int64)i < 0x12; ++i )
6     {
7         if ( byte_140003020[* (unsigned __int8 *)(a1 + i)] != byte_140003000[i] )
8             return 0LL;
9     }
10    return 1LL;
11 }

```

63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C6
B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	19
04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
09	83	2C	1A	1B	6E	5A	0A	52	3B	D6	B3	29	E3	2F	84
53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
70	3E	B5	66	48	03	F6	0E	61	35	57	89	86	C1	1D	9E
E1	F8	98	11	69	D9	8E	04	9B	1E	87	E9	CE	55	2D	DF
8C	A1	89	0D	BE	F6	42	68	41	99	2D	0E	B0	54	BB	1F

```
00 4D 51 50 EF FB C3 CF 92 45 4D CF F5 04 40 50
43 63 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

다음과 같은 C++ 코드를 만들어 각각 byte_140003020의 값과 byte_140003000의 값을 모두 차례로 입력하니 Flag를 얻을 수 있었다.

```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5  int main() {
6      string answer;
7      int n=0;
8      char temp;
9      vector<int> v1(256);
10     vector<int> v2(18);
11
12     for(int i=0; i<256; i++)
13         cin >> hex >> v1[i];
14     for(int i=0; i<18; i++)
15         cin >> hex >> v2[i];
16
17     for (int i = 0; i < 18; i++) {
18         while(1) {
19             if(v1[n]==v2[i])
20                 break;
21             else
22                 n++;
23         }
24         temp = (char)n;
25         answer += temp;
26         n = 0;
27     }
28     cout << answer;
29     return 0;
30 }
```


rev-basic-7

- 문제 설명

생략

- 풀이 과정

ROL 연산은 비트를 왼쪽으로 회전시키는 연산이라고 한다. 주어진 문제는 31번에 걸쳐 `a1[i]`을 `i&7`번 ROL 연산하여 얻은 값과 `i`를 XOR한 값이 `byte_140003000[i]`와 같아야 한다.

```
1 __int64 __fastcall sub_140001000(__int64 a1)
2 {
3     int i; // [rsp+0h] [rbp-18h]
4
5     for ( i = 0; (unsigned __int64)i < 0x1F; ++i )
6     {
7         if ( (i ^ (unsigned __int8)__ROL1__(*(__BYTE *) (a1 + i), i & 7)) != byte_140003000[i] )
8             return 0LL;
9     }
10    return 1LL;
11 }
```

Flag를 구하려면 XOR의 역연산은 XOR이므로 `byte_140003000[i]`과 `i`를 역연산한 후 비트를 오른쪽으로 회전시키는 ROR 연산을 적용시켜야 한다.

ROR 연산을 함수로 구현시킨 C++ 코드를 만들어 `byte_140003000`의 모든 값을 차례로 입력하니 Flag를 얻을 수 있었다.

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  char ror(unsigned char temp, int i) {
6      if((i&7) == 0)
7          return temp;
8      return (char)((temp >> (i&7)) | (temp << (8 - (i&7))));
9  }
10
11 int main() {
12     string answer;
13     int num;
14     char temp;
15     for (int i = 0; i < 31; i++) {
16         cin >> hex >> num;
17         temp = (char)(num^i);
18         answer += ror(temp, i);
19     }
20     cout << answer;
21     return 0;
22 }
```

rev-basic-8

- 문제 설명
생략

- 풀이 과정

21번에 걸쳐 (unsigned_int8)(-5*a1[i])과 byte_140003000[i]가 일치해야 한다.

```
1 __int64 __fastcall sub_140001000(__int64 a1)
2 {
3     int i; // [rsp+0h] [rbp-18h]
4
5     for ( i = 0; (unsigned __int64)i < 0x15; ++i )
6     {
7         if ( (unsigned __int8)(-5 * *(_BYTE *) (a1 + i)) != byte_140003000[i] )
8             return 0LL;
9     }
10    return 1LL;
11 }
```

(unsigned_int8)(-5*a1[i])은 1) unsigned_int8이므로 하위 8비트, 즉 256으로 나눈 나머지 값이 된다. 2) a1[i]에 -5를 곱한 후 256으로 나눈 나머지는 256-5=251을 곱한 후 나머지와 같다.

이 해석을 바탕으로 역연산을 구하려고 했는데 모듈러 연산의 역연산을 모르겠어서 검색하니 역원을 찾아서 곱하면 된다고 한다. 아래 코드로 역원을 찾으니 51이 나온다. $(251 \times 51) \bmod 256 = 1$ 이므로 검산도 맞다.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int main() {
6     int answer, k=1;
7     while(1) {
8         if((251*k)%256==1)
9             break;
10        else
11            k++;
12    }
13    cout << k;
14    return 0;
15 }
```

역원을 51로 놓고 아래와 같은 C++ 코드를 만들어 Flag를 찾을 수 있었다.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int main() {
6     string answer;
7     int num;
8     char temp;
9     for (int i = 0; i < 21; i++) {
10        cin >> hex >> num;
11        temp = (char)((num*51)%256);
12        answer += temp;
13    }
14    cout << answer;
15    return 0;
16 }
```

rev-basic-9

- 문제 설명

생략

- 풀이 과정

main함수의 모습이다. sub_140001000(v4)를 만족해야 한다는 것이 보인다.

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char v4[256]; // [rsp+20h] [rbp-118h] BYREF
4
5     memset(v4, 0, sizeof(v4));
6     sub_1400012E0("Input : ", argv, envp);
7     sub_140001340("%256s", v4);
8     if ( (unsigned int)sub_140001000(v4) )
9         puts("Correct");
10    else
11        puts("Wrong");
12    return 0;
13 }
```

이전과 다르게 단순 비교가 아니라서 하나씩 해석하기로 했다.

```
1 _BOOL8 __fastcall sub_140001000(const char *a1)
2 {
3     int i; // [rsp+20h] [rbp-18h]
4     int v3; // [rsp+24h] [rbp-14h]
5
6     v3 = strlen(a1);
7     if ( (v3 + 1) % 8 )
8         return 0LL;
9     for ( i = 0; i < v3 + 1; i += 8 )
10        sub_1400010A0(&a1[i]);
11    return memcmp(a1, &unk_140004000, 0x19uLL) == 0;
12 }
```

1) v3이라는 변수는 a1(입력값)의 길이를 갖는다.

2) (v3+1)을 8로 나눈 나머지가 0이 아니라면 0을 반환한다. 따라서 v3+1은 8의 배수가 되어야 한다.

3) a1[i]값을 sub_1400010A0 함수에 넣어 변환하는 것으로 추정된다.

4) (아마도 변환이 끝난) a1을 unk_140004000과 25바이트 비교해 전부 같아야 한다.

sub_1400010A0 함수도 복잡해보이니 하나씩 해석하기로 한다.

```
1 __int64 __fastcall sub_1400010A0(unsigned __int8 *a1)
2 {
3     __int64 result; // rax
4     unsigned __int8 v2; // [rsp+0h] [rbp-48h]
5     int j; // [rsp+4h] [rbp-44h]
6     int i; // [rsp+8h] [rbp-40h]
7     char v5[16]; // [rsp+10h] [rbp-38h] BYREF
8
9     strcpy(v5, "I_am_KEY");
10    result = *a1;
11    v2 = *a1;
12    for ( i = 0; i < 16; ++i )
13    {
14        for ( j = 0; j < 8; ++j )
15        {
16            v2 = __ROR1__(a1[((_BYTE)j + 1) & 7] + byte_140004020[(unsigned __int8)v5[j] ^ v2], 5);
17            a1[((_BYTE)j + 1) & 7] = v2;
18        }
19        result = (unsigned int)(i + 1);
20    }
21    return result;
22 }
```

- 1) v5에는 "I_am_KEY"가 복사되어 있다.
- 2) v2는 입력받은 a1[0]값이다.
- 3) byte_140004020에서 v5[j]와 v2를 XOR한 값을 색인으로 해 값을 찾아 a1[(j+1)&7]한 값과 더한 뒤
- 4) 오른쪽으로 비트를 회전시키는 ROR 연산을 5비트한다.
- 5) 그 값을 a1[(j+1)&7]에 넣는다.

이를 역연산해서 Flag를 구해야 한다. 8바이트씩 분리한 후 뒤에서부터 역순으로 각 값을 ROL 연산 5비트를 한 뒤 XOR 한 값을 빼는 과정을 반복하는 C++ 코드를 만들었다.

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  using namespace std;
5
6  unsigned char rol(unsigned char v, int r) {
7      r = r&7;
8      if (r == 0)
9          return v;
10     return (unsigned char)((v << r) | (v >> (8 - r)));
11 }
12
13 int main() {
14     string answer, key = "I_am_KEY";
15     vector<unsigned char> v1 = {99,124,119,123,242,107,111,197,48,1,103,43,254,215,171,118,202,130,201,125,250,89,71,240,173,212,162,175,156,164};
16     vector<unsigned char> v2 = {126,125,154,139,37,45,213,61,3,43,56,152,39,159,79,188,42,121,0,125,196,42,79,88};
17
18     for (size_t n = 0; n < v2.size(); n += 8) {
19         vector<unsigned char> v3(v2.begin()+n, v2.begin()+n+8);
20
21         for (int i = 15; i >= 0; i--) {
22             for (int j = 7; j >= 0; j--) {
23                 int k = (j+1) & 7;
24                 unsigned char c = v3[j];
25                 unsigned char t = v1[(unsigned char)(key[j] ^ c)];
26                 v3[k] = (unsigned char)(rol(v3[k], 5) - t);
27             }
28         }
29
30         for (auto it : v3)
31             if (it != 0)
32                 answer += (char)it;
33     }
34
35     cout << answer;
36     return 0;
37 }

```